

Linear Programming for Traffic Engineering

Antonio Carzaniga

April 15, 2026

1 Linear Programming: A Very Quick Introduction

Linear programming¹ is a very general and versatile method to model and solve optimization problems. Let's start with one such problem as an example. We will see later what all of this has to do with networking.

Say you run an ice-cream shop. You know you can sell all the ice cream you make, but you can choose to make different flavors of ice cream that require different ingredients. This week you are thinking about making *fruity-chocolate* and *nutty-vanilla* ice cream, and you'd like to decide how much to make of each flavor.

According to your recipe, in order to make a liter of fruity-chocolate you need 250g of sugar, 500ml of milk, 20g of nuts, 200g of cocoa, and 100g of raisins. Instead, to make a liter of nutty-vanilla ice cream, you need 350g of sugar, 450ml of milk, 200g of nuts, and 40g of raisins.

You have 1000g of sugar, 2 liters of milk, 500g of nuts, 300g of raisins, and 600g of cocoa. Again, assuming you can sell all the ice cream you make, and also that you sell the two flavors for the same price per liter, how much should you make of each flavor to maximize your profits?

This is a classic linear programming model. Let's see what you need to put together a linear program in general, and in particular for your ice cream shop.

Linear Program

1. Identify your *control variables*

- c quantity of fruity chocolate (liters)
- v quantity of nutty vanilla (liters)

2. Specify your *objective function*

maximize $c + v$

3. Specify your *constraints or limits*

- $250c + 350v \leq 1000$ (sugar)
- $500c + 450v \leq 2000$ (milk)
- $20c + 200v \leq 500$ (nuts)
- $100c + 40v \leq 300$ (raisins)
- $200c \leq 600$ (cocoa)

¹The term "programming" here does not refer to what we usually call computer programs today. (The same is true for the generic algorithmic technique called "dynamic programming.") A less common but more appropriate term is linear *optimization*. The use of the term programming is rooted in the applications of these optimization techniques to logistics and therefore "programming" problems.

The first step is to identify your control variables. This is what you have to decide upon. In our ice-cream optimization problem you are asked to decide how much you should make of each flavor, so your control variables are the amount of fruity chocolate c and the amount of nutty vanilla v .

Next, you must define an objective function. This must be a *linear* function of your control variables. In the example problem, you are told to maximize your profits, and since the sale price of the two flavors is the same, then the objective function is simply $c + v$. If you had different prices, say 7 for fruity chocolate and 8 for nutty vanilla, you would still be able to define a linear objective function for your optimization problem as $7c + 8v$.

Third and last step, you must specify all the necessary constraints of the problem as a function of your control variables. Once again, these must be *linear* constraints. In the ice-cream problem, the constraints are given by the limited amounts of ingredients. For example, since the recipes for fruity chocolate and nutty vanilla call for 250 and 350 grams of sugar, respectively, and since you have a total of 1000 grams of sugar, then that poses a constraint $250c + 350v \leq 1000$. And similarly you have constraints for all other ingredients.

1.1 General LP Formulation

In general, given a set of variables $x_1, x_2, \dots, x_n$, you define your optimization objective, meaning some quantity that you want to maximize or minimize, as a linear combination of the variables $x_1, \dots, x_n$.

$$\text{maximize } c_1x_1 + c_2x_2 + \dots + c_nx_n$$

The optimization is subject to a series of linear constraints:

$$\begin{aligned} a_{1,1}x_1 + a_{1,2}x_2 + \dots + a_{1,n}x_n &\leq b_1 \\ a_{2,1}x_1 + a_{2,2}x_2 + \dots + a_{2,n}x_n &\leq b_2 \\ &\vdots \\ a_{m,1}x_1 + a_{m,2}x_2 + \dots + a_{m,n}x_n &\leq b_m \end{aligned}$$

Again, these are all linear equations—that's why the technique is called *linear* programming—so the equations can be expressed more compactly in matrix form. Let $\mathbf{x} = [x_1 x_2 \dots x_n]^T$ be the vector of control variables; letting $\mathbf{c} = [c_1 c_2 \dots c_n]$ be the vector of coefficients for the objective function; letting $\mathbf{A}$ be an $m \times n$ matrix of constraint coefficients; and letting $\mathbf{b} = [b_1 b_2 \dots b_m]^T$ be a vector of limits, we simply write the linear program in matrix form like so:

$$\begin{aligned} &\text{maximize } \mathbf{c}\mathbf{x} \\ &\text{subject to } \mathbf{A}\mathbf{x} \leq \mathbf{b} \end{aligned}$$

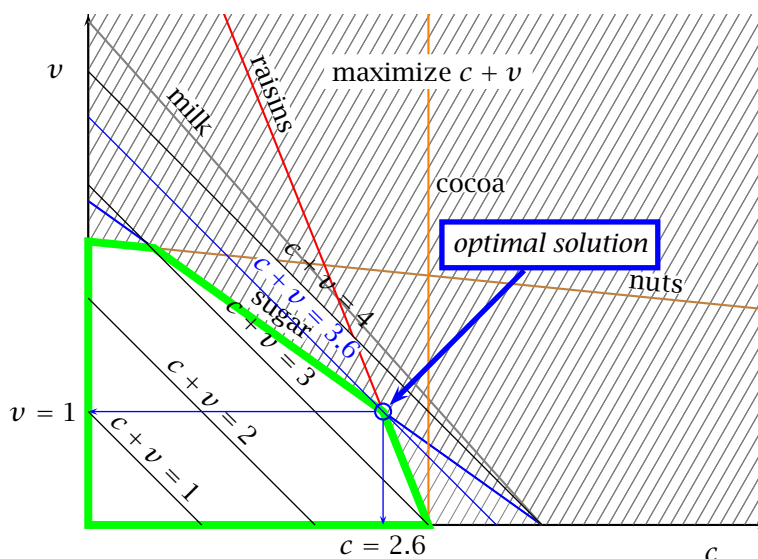
We often also require that the control variables be non-negative, so we have the following general linear program

$$\begin{aligned} &\text{maximize } \mathbf{c}\mathbf{x} \\ &\text{subject to } \mathbf{A}\mathbf{x} \leq \mathbf{b} \\ &\quad \text{and } \mathbf{x} \geq \mathbf{0} \end{aligned}$$

1.2 Geometric Interpretation

Let's consider the geometric interpretation of a linear program. The variables define the space of interest. Within that space, the constraints define subspaces of valid solutions.

In the ice-cream example, we consider the 2D plane where points have coordinates (c, v) . Since c and v are non-negative quantities, the space is the upper-right sector of the 2D plane. Within that space, the constraints represent lines that further limit the space of valid solutions. Considering all constraints, plus the implicit constraints that c and v are non-negative, we are left with a convex polygon. Within this polygon, we are supposed to find the point that maximizes the objective function $c + v$.



2 LP Formulation of Flow Problems

So far so good. We now know how to write and also how to give a geometrical interpretation of a linear program.

But what does all this have to do with networking? The connection is in the fact that linear programming is a powerful modeling and optimization technique that is particularly suitable for interesting networking problems. More specifically, *traffic engineering* problems.

Basic routing protocols build shortest paths. This makes sense if we want to minimize latency and total traffic. However, shortest paths also tend to create congestion. This can be understood by thinking about road traffic. Take for example a city. If every vehicle were to choose the shortest path to go from their starting point to their destination, most paths would go through the center of the city, which would therefore create congestion for the streets in that central area. The same is true in networking. Deterministic, shortest-path routing tends to cause congestion.

Traffic engineering refers to the methods one can use to direct network traffic so as to satisfy some optimization goal, typically to minimize link or node utilization and therefore congestion while achieving some transmission objectives, such as achieving some given flow rates. Or conversely, the goal could be to maximize flow rates while maintaining link utilization under some given thresholds.

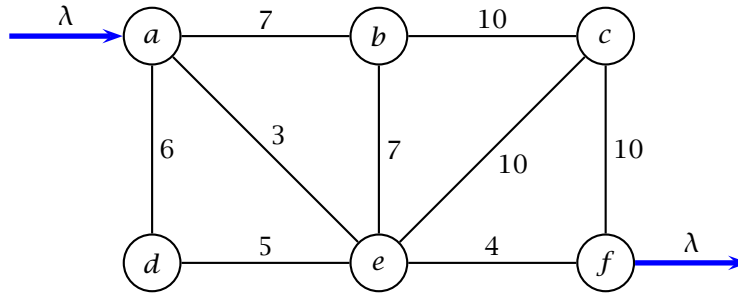
We now describe a series of such traffic engineering or “flow” problems.

2.1 Maximum Flow

We consider a network with fixed link capacities $C(u, v)$ for each link (u, v) . We start with a most basic scenario in which we are given one flow requirement $f = (s, t, F)$, which specifies that we want to transmit F bytes from source node s to target node t . In this case, we want to minimize the completion time for flow f , which also means that we want to maximize the total, effective transmission rate or “goodput” for f while not exceeding the specified link capacities.

In this first formulation, we assume that traffic can be split and merged as needed. This is the classic *max flow* combinatorial problem for which there are well known and efficient algorithms. Here we use a linear program to model and then solve the problem.

In order to define a linear program, we have to define the control variables, the objective function, and the constraints. At first, it might seem difficult to formulate our flow problem as a linear program. The problem requires us to decide *where* to route traffic. However, the control variables in a linear program typically express *how much* we assign to one or another control. So, the trick is to use these variables to express how traffic is routed one way or another. As we will see with a progression of problems, this is the essence of modeling flow problems as linear programs. In fact, it is the essence of modeling any problem as a linear program!



Variables: x_{uv} for each link (u, v) : amount of traffic flowing through the (u, v) link; plus the input/output flow λ .

Objective: maximize λ

Constraints: $-C(u, v) \leq x_{uv} \leq C(u, v)$ for each link (u, v) : traffic must obey link capacities

$\sum_{v \in \text{adj}(u)} x_{uv} = 0$ for each node u : net flow in and out of u is zero, including global input and output links.

In this case, we define a variable x_{uv} for each link (u, v) to represent the amount of traffic flowing through link (u, v) per unit time. Notice that the problem is such that we will never want to route traffic in both directions on any link. This is because there is only one flow, so transporting data from u to v and then back from v to u amounts to introducing a wasteful cycle. This allows us to use a single variable x_{uv} for each link (u, v) , which will be positive when traffic flows from u to v and negative when traffic goes from v to u . This also means that x_{vu} is really the same as x_{uv} , only with the opposite sign. Therefore we only use one of the two variables. We also define a variable λ representing the intensity of the incoming flow into source node s and out of target node t . Therefore, our objective is to maximize λ .

The network topology constrains these flow variables. Each per-link flow must be within the corresponding link capacity. Thus we require that $-C(u, v) \leq x_{uv} \leq C(u, v)$ for each link (u, v) . Furthermore, each node constrains all the adjacent flows. In particular, the net flow going through each node must be zero, since in this model data does not accumulate or originate at any node.

In summary, we obtain the following linear program:

$$\begin{array}{ll}
\text{maximize } \lambda & \\
\text{subject to } -C(u, v) \leq x_{uv} \leq C(u, v) & \text{for all links } (u, v) \\
\sum_{v \in \text{adj}(s)} x_{sv} + \lambda = 0 & \text{for source node } s \\
\sum_{v \in \text{adj}(t)} x_{tv} - \lambda = 0 & \text{for target node } t \\
\sum_{v \in \text{adj}(u)} x_{uv} = 0 & \text{for all other nodes } u
\end{array}$$

We can then use a solver to obtain an optimal solution. In this and all the following examples, we use the GNU Linear Programming Kit². All solvers provide a programmatic way of defining linear programs. However, here we show and use problem definitions coded in the text-based and quite readable CPLEX LP standard format.

```

Maximize
obj: lambda

Subject to
a: lambda - x_ab - x_ae - x_ad = 0
b: x_ab - x_be - x_bc = 0
c: x_bc - x_ce - x_cf = 0
d: x_ad - x_de = 0
e: x_de + x_ae + x_be + x_ce - x_ef = 0
f: x_ef + x_cf - lambda = 0

Bounds
-7 <= x_ab <= 7
-3 <= x_ae <= 3
-6 <= x_ad <= 6
-10 <= x_bc <= 10
-7 <= x_be <= 7
-10 <= x_ce <= 10
-10 <= x_cf <= 10
-5 <= x_de <= 5
-4 <= x_ef <= 4

End

```

Notice once again that we define a single variable for each edge. For example, we define a single variable x_{ab} (x_{ab}) to indicate the flow on edge (a, b) , which we consider as an outgoing flow from a and as an incoming flow into b . Thus the flow-balance constraints for node a (labeled “a:” in the problem definition) subtracts x_{ab} while the corresponding constraint for node b adds x_{ab} .

We now run the solver. Here we use the `glpsol` command-line tool. Assuming the input problem definition is stored in a file called `maxflow_single.lp`, we run the following command:

```
$ glpsol -o maxflow_single.txt --lp maxflow_single.lp
```

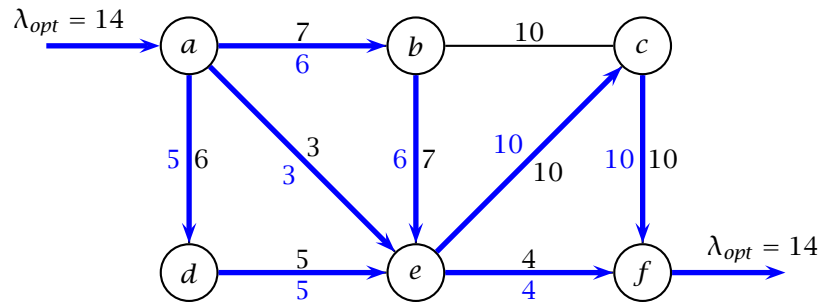
²<https://www.gnu.org/software/glpk/>

The solver logs some diagnostic messages on the terminal and then solves the problem, writing the solution into the indicated text file `maxflow_single.txt`:

Problem:						
Rows: 6						
Columns: 10						
Non-zeros: 20						
Status: OPTIMAL						
Objective: obj = 14 (MAXimum)						
No.	Row name	St	Activity	Lower bound	Upper bound	Marginal
1	a	NS	0	0	=	1
2	b	NS	0	0	=	1
3	c	NS	0	0	=	1
4	d	NS	0	0	=	1
5	e	NS	0	0	=	1
6	f	B	0	0	=	
No.	Column name	St	Activity	Lower bound	Upper bound	Marginal
1	lambda	B	14	0		
2	x _{ab}	B	6	-7	7	
3	x _{ae}	NU	3	-3	3	< eps
4	x _{ad}	B	5	-6	6	
5	x _{be}	B	6	-7	7	
6	x _{bc}	B	0	-10	10	
7	x _{ce}	NL	-10	-10	10	< eps
8	x _{cf}	NU	10	-10	10	1
9	x _{de}	NU	5	-5	5	< eps
10	x _{ef}	NU	4	-4	4	1
Karush-Kuhn-Tucker optimality conditions:						
KKT.PE: max.abs.err = 0.00e+00 on row 0						
max.rel.err = 0.00e+00 on row 0						
High quality						
KKT.PB: max.abs.err = 0.00e+00 on row 0						
max.rel.err = 0.00e+00 on row 0						
High quality						
KKT.DE: max.abs.err = 0.00e+00 on column 0						
max.rel.err = 0.00e+00 on column 0						
High quality						
KKT.DB: max.abs.err = 0.00e+00 on row 0						
max.rel.err = 0.00e+00 on row 0						
High quality						
End of output						

The key component of the output file is the list of control variables (columns) with their optimal assignments, from which we can figure out the overall resulting flows. The same variables can then be used in each node to route traffic. For example, variables $x_{ab} = 6$, $x_{ad} = 5$, and $x_{ae} = 3$ that define the allocation of capacities of the (a, b) , (a, d) , and (a, e) , respectively, also define the proportion of the

incoming flow into node a that must be routed through b , d , and e , respectively. In particular, node a must route $6/14 = 43\%$ of the traffic towards b , $5/14 = 36\%$ towards d , and $3/14 = 21\%$ towards e .



2.2 Multiple Flows

So far, we have considered the maximization of a *single* flow. What happens if we want to maximize multiple flows?

We first consider a simple extension of the single-flow problem in which we have multiple source and target nodes but we can still consider all the flows as carrying effectively the same traffic. This model makes sense in concrete problems where we need to transport a single commodity, such as food or energy, from some sources to some destinations. In such cases, what matters for the receivers is the *amount* they receive, not where that amount of the commodity came from. By contrast, later we see a variant of this problem with *independent* flows, which is appropriately called multi-commodity flow optimization. The single-commodity model that we start with is arguably less relevant for computer networking, where flows are typically defined by and bound to both the source and the destination. Still, the model we develop for this problem serves as a basis to develop other more general and powerful linear programs with various optimization goals.

All we have to do to model additional sources and targets is to add the corresponding input and output variables to indicate all the incoming and outgoing flows. In the example below we use two inputs, i_1 and i_2 , and two outputs, o_1 and o_2 , and we maximize the total goodput $i_1 + i_2 = o_1 + o_2$.

```

Maximize
obj: i1 + i2

Subject to
a: i1 - x_ab - x_ae - x_ad = 0
b: x_ab - x_be - x_bc = 0
c: x_bc - x_ce - x_cf - o1 = 0
d: i2 + x_ad - x_de = 0
e: x_de + x_ae + x_be + x_ce - x_ef = 0
f: x_ef + x_cf - o2 = 0

Bounds
-7 <= x_ab <= 7
-3 <= x_ae <= 3
-6 <= x_ad <= 6
-10 <= x_bc <= 10
-7 <= x_be <= 7
-10 <= x_ce <= 10

```

```

-10 <= x_cf <= 10
-5 <= x_de <= 5
-4 <= x_ef <= 4
End

```

In this setting, in which we effectively have a single type of traffic, we can also imagine different types of goals. For example, we still want to maximize traffic but we might want to do that with some sort of fairness. A good fairness principle is to maximize the least flows. So, in the next example, we change the model to maximize the *minimal* input flow, as opposed to the total flow. However, we can not simply declare the objective function to be the minimum between two or more variables, since $\min(x, y)$ is not a linear function of x and y . The question, then, is how to express that with a linear program.

There are no universal techniques to model problems and objectives with linear programs. However, a common approach that is very often effective is to introduce auxiliary variables. In this case, we want to maximize the minimum among a certain number of quantities, namely i_1 and i_2 in our example. We therefore introduce a variable m that we set as the objective function, and that we constrain by the values of i_1 and i_2 . Thus we add the two constraints $m \leq i_1$ and $m \leq i_2$. Notice that those constraints by themselves only define m to be less than or equal to the minimum between i_1 and i_2 . However, since m is not constrained in other ways, and since the objective is to maximize m , the solution of the linear program will indeed equate m with the minimum between i_1 and i_2 .

```

Maximize
obj: m

Subject to
min1: m - i1 <= 0
min2: m - i2 <= 0

a: i1 - x_ab - x_ae - x_ad = 0
b: x_ab - x_be - x_bc = 0
c: x_bc - x_ce - x_cf - o1 = 0
d: i2 + x_ad - x_de = 0
e: x_de + x_ae + x_be + x_ce - x_ef = 0
f: x_ef + x_cf - o2 = 0

Bounds
-7 <= x_ab <= 7
-3 <= x_ae <= 3
-6 <= x_ad <= 6
-10 <= x_bc <= 10
-7 <= x_be <= 7
-10 <= x_ce <= 10
-10 <= x_cf <= 10
-5 <= x_de <= 5
-4 <= x_ef <= 4
End

```

Still in this same single-traffic setting, we might want to specify yet other optimization goals. So far, we modeled programs that maximize throughput while keeping link usage below their capacity. That is, the desire to obtain high throughput is expressed as part of the objective function, while link-capacity limits are expressed as part of the constraints. Now we might want to do the opposite, that is, we might want to satisfy one or more fixed flow demands, expressing them as constraints, while at the same time minimizing link usage as a new objective function.

Notice that this type of model is very relevant to traffic engineering (apart from the fact that it is a

single-commodity flow model) since it corresponds to the case in which we have fixed or anyway more or less regular traffic demands, and our goal is to avoid or anyway minimize congestion. Even when we don't have fixed flow demands, we most often transmit flows using adaptive transport protocols, such as TCP, that try to maximize throughput anyway. As a result, we have a network in which engineering of traffic done at the global high-level with our linear programs guarantees a balanced use of the network, while at the same time individual flow controllers (TCP) optimize throughput.

So, back to modeling with linear programs, we want to minimize congestion. We can express this optimization objective in at least three ways. The example below minimizes the *total* link usage. Notice that our control variables x_{uv} represent *directed* flows, which might be positive or negative depending on the direction. So, we can not simply add those variables to compute the total usage. Instead, our objective function is the sum $\sum_{(u,v) \in E} |x_{uv}|$. Once again, we need to model a non-linear function (the absolute value), and once again we do that by introducing appropriate auxiliary variables, in this case one per link to model $|x_{uv}|$.

Minimize

obj: $x_{ab_abs} + x_{ae_abs} + x_{ad_abs} + x_{bc_abs} + x_{be_abs} + x_{ce_abs} + x_{cf_abs} + x_{de_abs} + x_{ef_abs}$

Subject to

demands: $i1 + i2 = 10$

a: $i1 - x_{ab} - x_{ae} - x_{ad} = 0$

b: $x_{ab} - x_{be} - x_{bc} = 0$

c: $x_{bc} - x_{ce} - x_{cf} - o1 = 0$

d: $i2 + x_{ad} - x_{de} = 0$

e: $x_{de} + x_{ae} + x_{be} + x_{ce} - x_{ef} = 0$

f: $x_{ef} + x_{cf} - o2 = 0$

ab_abs_p: $x_{ab} - x_{ab_abs} \leq 0$

ab_abs_n: $x_{ab} + x_{ab_abs} \geq 0$

ae_abs_p: $x_{ae} - x_{ae_abs} \leq 0$

ae_abs_n: $x_{ae} + x_{ae_abs} \geq 0$

ad_abs_p: $x_{ad} - x_{ad_abs} \leq 0$

ad_abs_n: $x_{ad} + x_{ad_abs} \geq 0$

bc_abs_p: $x_{bc} - x_{bc_abs} \leq 0$

bc_abs_n: $x_{bc} + x_{bc_abs} \geq 0$

be_abs_p: $x_{be} - x_{be_abs} \leq 0$

be_abs_n: $x_{be} + x_{be_abs} \geq 0$

ce_abs_p: $x_{ce} - x_{ce_abs} \leq 0$

ce_abs_n: $x_{ce} + x_{ce_abs} \geq 0$

cf_abs_p: $x_{cf} - x_{cf_abs} \leq 0$

cf_abs_n: $x_{cf} + x_{cf_abs} \geq 0$

de_abs_p: $x_{de} - x_{de_abs} \leq 0$

de_abs_n: $x_{de} + x_{de_abs} \geq 0$

ef_abs_p: $x_{ef} - x_{ef_abs} \leq 0$

ef_abs_n: $x_{ef} + x_{ef_abs} \geq 0$

Bounds

$-7 \leq x_{ab} \leq 7$

$-3 \leq x_{ae} \leq 3$

$-6 \leq x_{ad} \leq 6$

$-10 \leq x_{bc} \leq 10$

$-7 \leq x_{be} \leq 7$

$-10 \leq x_{ce} \leq 10$

$-10 \leq x_{cf} \leq 10$

```

-5 <= x_de <= 5
-4 <= x_ef <= 4
End

```

Notice that minimizing the *total* link usage is analogous to a shortest-path routing scheme. (Can you argue why?) However, since we are interested in spreading traffic evenly so as to reduce congestion, a better objective would be to minimize the *maximal* link usage, as we do in the example below. Notice that here we can do away with the per-link absolute-value auxiliary variables, since we introduce an auxiliary variable x_{max} that subsumes all of them.

```

Minimize
obj: x_max

Subject to
demands: i1 + i2 = 10

a: i1 - x_ab - x_ae - x_ad = 0
b: x_ab - x_be - x_bc = 0
c: x_bc - x_ce - x_cf - o1 = 0
d: i2 + x_ad - x_de = 0
e: x_de + x_ae + x_be + x_ce - x_ef = 0
f: x_ef + x_cf - o2 = 0

ab_abs_p: x_ab - x_max <= 0
ab_abs_n: x_ab + x_max >= 0
ae_abs_p: x_ae - x_max <= 0
ae_abs_n: x_ae + x_max >= 0
ad_abs_p: x_ad - x_max <= 0
ad_abs_n: x_ad + x_max >= 0
bc_abs_p: x_bc - x_max <= 0
bc_abs_n: x_bc + x_max >= 0
be_abs_p: x_be - x_max <= 0
be_abs_n: x_be + x_max >= 0
ce_abs_p: x_ce - x_max <= 0
ce_abs_n: x_ce + x_max >= 0
cf_abs_p: x_cf - x_max <= 0
cf_abs_n: x_cf + x_max >= 0
de_abs_p: x_de - x_max <= 0
de_abs_n: x_de + x_max >= 0
ef_abs_p: x_ef - x_max <= 0
ef_abs_n: x_ef + x_max >= 0

Bounds
-7 <= x_ab <= 7
-3 <= x_ae <= 3
-6 <= x_ad <= 6
-10 <= x_bc <= 10
-7 <= x_be <= 7
-10 <= x_ce <= 10
-10 <= x_cf <= 10
-5 <= x_de <= 5
-4 <= x_ef <= 4
End

```

In yet another example, we minimize the maximal *relative* usage, meaning relative to link capacity (i.e., percent link utilization), which is even more interesting for congestion.

Minimize

\ *min max percent capacity*

obj: pc_max

Subject to

demands: i1 + i2 = 8

ab_max: x_ab_abs - 7 pc_max <= 0
ae_max: x_ae_abs - 3 pc_max <= 0
ad_max: x_ad_abs - 6 pc_max <= 0
bc_max: x_bc_abs - 10 pc_max <= 0
be_max: x_be_abs - 7 pc_max <= 0
ce_max: x_ce_abs - 10 pc_max <= 0
cf_max: x_cf_abs - 10 pc_max <= 0
de_max: x_de_abs - 5 pc_max <= 0
ef_max: x_ef_abs - 5 pc_max <= 0

\ *flow balance constraints, for each node*

a: i1 - x_ab - x_ae - x_ad = 0
b: x_ab - x_be - x_bc = 0
c: x_bc - x_ce - x_cf - o1 = 0
d: i2 + x_ad - x_de = 0
e: x_de + x_ae + x_be + x_ce - x_ef = 0
f: x_ef + x_cf - o2 = 0

ab_abs_p: x_ab - x_ab_abs <= 0
ab_abs_n: x_ab + x_ab_abs >= 0
ae_abs_p: x_ae - x_ae_abs <= 0
ae_abs_n: x_ae + x_ae_abs >= 0
ad_abs_p: x_ad - x_ad_abs <= 0
ad_abs_n: x_ad + x_ad_abs >= 0
bc_abs_p: x_bc - x_bc_abs <= 0
bc_abs_n: x_bc + x_bc_abs >= 0
be_abs_p: x_be - x_be_abs <= 0
be_abs_n: x_be + x_be_abs >= 0
ce_abs_p: x_ce - x_ce_abs <= 0
ce_abs_n: x_ce + x_ce_abs >= 0
cf_abs_p: x_cf - x_cf_abs <= 0
cf_abs_n: x_cf + x_cf_abs >= 0
de_abs_p: x_de - x_de_abs <= 0
de_abs_n: x_de + x_de_abs >= 0
ef_abs_p: x_ef - x_ef_abs <= 0
ef_abs_n: x_ef + x_ef_abs >= 0

Bounds

-7 <= x_ab <= 7
-3 <= x_ae <= 3
-6 <= x_ad <= 6
-10 <= x_bc <= 10
-7 <= x_be <= 7
-10 <= x_ce <= 10
-10 <= x_cf <= 10
-5 <= x_de <= 5
-4 <= x_ef <= 4

End

2.3 Multi-Commodity Flows

Up to now, we have considered flows for which we can direct whichever portions of the input flow from a source to any number of destinations, and vice-versa a destination can take different amounts of flows from any number of sources. This makes sense, for instance, for a transportation network when the objective is to transfer a single commodity. But that is most likely not what we have in a computer network.

In general, when we have two flows $f_1 = (s_1, t_1, F_1)$ and $f_s = (s_2, t_2, F_2)$, we can not mix the two flows. So, we have to change the model. The way we model separate flows is, once again, by adding variables. Specifically, we add per-link flow variables *for all flows*. In the following example we have two flows, which we denote with variables x and y , respectively. Thus x_{uv} is the portion of the x flow passing through the (u, v) edge, which is distinct from y_{uv} , which is the portion of the y flow going from u to v .

Maximize

obj: $x + y$

\ *Flows:*

\ $X: A, B \rightarrow F$

\ $Y: D \rightarrow C$

Subject to

\ *flow balance constraints: for each flow, for each node*

$x_total: x - x_1 - x_2 = 0$

$xa: x_1 - x_{ab} - x_{ae} - x_{ad} = 0$

$xb: x_2 + x_{ab} - x_{be} - x_{bc} = 0$

$xc: x_{bc} - x_{ce} - x_{cf} = 0$

$xd: x_{ad} - x_{de} = 0$

$xe: x_{de} + x_{ae} + x_{be} + x_{ce} - x_{ef} = 0$

$xf: x_{ef} + x_{cf} - x = 0$

$ya: y_{ab} + y_{ae} + y_{ad} = 0$

$yb: y_{ab} - y_{be} - y_{bc} = 0$

$yc: y_{bc} - y_{ce} - y_{cf} - y = 0$

$yd: y + y_{ad} - y_{de} = 0$

$ye: y_{de} + y_{ae} + y_{be} + y_{ce} - y_{ef} = 0$

$yf: y_{ef} + y_{cf} = 0$

$x_{ab_abs_p}: x_{ab} - x_{ab_abs} \leq 0$

$x_{ab_abs_n}: x_{ab} + x_{ab_abs} \geq 0$

$x_{ae_abs_p}: x_{ae} - x_{ae_abs} \leq 0$

$x_{ae_abs_n}: x_{ae} + x_{ae_abs} \geq 0$

$x_{ad_abs_p}: x_{ad} - x_{ad_abs} \leq 0$

$x_{ad_abs_n}: x_{ad} + x_{ad_abs} \geq 0$

$x_{bc_abs_p}: x_{bc} - x_{bc_abs} \leq 0$

$x_{bc_abs_n}: x_{bc} + x_{bc_abs} \geq 0$

$x_{be_abs_p}: x_{be} - x_{be_abs} \leq 0$

$x_{be_abs_n}: x_{be} + x_{be_abs} \geq 0$

$x_{ce_abs_p}: x_{ce} - x_{ce_abs} \leq 0$

$x_{ce_abs_n}: x_{ce} + x_{ce_abs} \geq 0$

$x_{cf_abs_p}: x_{cf} - x_{cf_abs} \leq 0$

$x_{cf_abs_n}: x_{cf} + x_{cf_abs} \geq 0$

$x_{de_abs_p}: x_{de} - x_{de_abs} \leq 0$

$x_{de_abs_n}: x_{de} + x_{de_abs} \geq 0$

$x_{ef_abs_p}: x_{ef} - x_{ef_abs} \leq 0$

$x_{ef_abs_n}: x_{ef} + x_{ef_abs} \geq 0$

```

y_ab_abs_p: y_ab - y_ab_abs <= 0
y_ab_abs_n: y_ab + y_ab_abs >= 0
y_ae_abs_p: y_ae - y_ae_abs <= 0
y_ae_abs_n: y_ae + y_ae_abs >= 0
y_ad_abs_p: y_ad - y_ad_abs <= 0
y_ad_abs_n: y_ad + y_ad_abs >= 0
y_bc_abs_p: y_bc - y_bc_abs <= 0
y_bc_abs_n: y_bc + y_bc_abs >= 0
y_be_abs_p: y_be - y_be_abs <= 0
y_be_abs_n: y_be + y_be_abs >= 0
y_ce_abs_p: y_ce - y_ce_abs <= 0
y_ce_abs_n: y_ce + y_ce_abs >= 0
y_cf_abs_p: y_cf - y_cf_abs <= 0
y_cf_abs_n: y_cf + y_cf_abs >= 0
y_de_abs_p: y_de - y_de_abs <= 0
y_de_abs_n: y_de + y_de_abs >= 0
y_ef_abs_p: y_ef - y_ef_abs <= 0
y_ef_abs_n: y_ef + y_ef_abs >= 0

```

```

ab: x_ab_abs + y_ab_abs <= 7
ae: x_ae_abs + y_ae_abs <= 3
ad: x_ad_abs + y_ad_abs <= 6
bc: x_bc_abs + y_bc_abs <= 10
be: x_be_abs + y_be_abs <= 7
ce: x_ce_abs + y_ce_abs <= 10
cf: x_cf_abs + y_cf_abs <= 10
de: x_de_abs + y_de_abs <= 5
ef: x_ef_abs + y_ef_abs <= 4

```

Bounds

```

\ -infinity <= x_uv <= +infinity

```

```

x_ab free
x_ae free
x_ad free
x_bc free
x_be free
x_ce free
x_cf free
x_de free
x_ef free

```

```

y_ab free
y_ae free
y_ad free
y_bc free
y_be free
y_ce free
y_cf free
y_de free
y_ef free

```

End

2.4 Routing Whole Flows

In the multi-commodity flow case, flows don't mix. However, we still allow ourselves to split each individual flow however we like. This may or may not be a reasonable assumption in concrete cases. For long-term flows made of many connections, the model is good. However, if you have very few connections or even a single connection, the model is completely inadequate.

The following examples model each flow as an independent unit. In these models we therefore use *integer* variables, or more specifically binary "indicator" variables. The idea is that a flow either goes through a link or it doesn't. Notice that the constraints and the objective functions remain linear functions of the control variables. However, the fact that the control variables are integral (or even just binary in this case) raises the complexity of solving the linear program to a completely different level. In fact, Linear Programming with real-valued variables (LP) can be solved in polynomial time and admits to very efficient solutions in practice. Instead, the integer variant of the problem, called Integer Linear Programming (ILP) or Mixed Integer Linear Programming (MILP) is NP-hard.

In the first example below, we model a single whole flow from source a to target f . We minimize the total number of hops for the flow, which corresponds to a minimal-hop path routing scheme.

Minimize

```
obj: x_ab + x_ba + x_ae + x_ea + x_ad + x_da + x_bc + x_cb + x_be + x_eb + x_ce + x_ec +  
      x_cf + x_fc + x_de + x_ed + x_ef + x_fe
```

Subject to

```
\ flow balance (flow: a --> f)  
a: x_da - x_ad + x_ea - x_ae + x_ba - x_ab = -1  
b: x_ab - x_ba + x_eb - x_be + x_cb - x_bc = 0  
c: x_bc - x_cb + x_ec - x_ce + x_fc - x_cf = 0  
d: x_ad - x_da + x_ed - x_de = 0  
e: x_de - x_ed + x_ae - x_ea + x_be - x_eb + x_ce - x_ec = 0  
f: x_ef - x_fe + x_cf - x_fc = 1
```

```
\ link capacities (flow demand = 4)
```

```
ab: 4 x_ab + 4 x_ba <= 7  
ae: 4 x_ae + 4 x_ea <= 3  
ad: 4 x_ad + 4 x_da <= 6  
bc: 4 x_bc + 4 x_cb <= 10  
be: 4 x_be + 4 x_eb <= 7  
ce: 4 x_ce + 4 x_ec <= 10  
cf: 4 x_cf + 4 x_fc <= 10  
de: 4 x_de + 4 x_ed <= 5  
ef: 4 x_ef + 4 x_fe <= 4
```

Binary

```
x_ab  
x_ba  
x_ae  
x_ea  
x_ad  
x_da  
x_bc  
x_cb  
x_be  
x_eb  
x_ce  
x_ec  
x_cf
```

```
x_fc  
x_de  
x_ed  
x_ef  
x_fe
```

End

In this simple whole-flow model, we use a control variable for each edge and for each flow, just as we did for the initial fractional max-flow model. However, in this case the variables are binary indicators. Thus x_{uv} does not represent the portion of the flow going through edge (u, v) but rather indicates whether or not that whole flow goes through edge (u, v) . Furthermore, in this model we choose binary variables, so we can not use a single variable to represent a bidirectional flow. We therefore use *two* indicator variables for each edge (and flow), one for each direction.

These indicator variables must still respect the same flow-balance constraints for each node, such that the net flow in and out of a node must be zero. The flow must also respect link capacities. However, since the indicator variables do not represent the intensity of the flow, we must multiply those variables by the flow intensity to write the capacity constraints. Thus for each link (u, v) , we write a constraint $Fx_{uv} + Fx_{vu} \leq C(u, v)$ for a given (constant) flow intensity F .

Notice that the constraints of this simple linear program do not in and of themselves ensure that paths are simple (cycle-free). This is not strictly necessary in this case since non-simple paths are not optimal according to the objective function. However, we can also explicitly constrain the model to represent simple paths. This can be done, as in the example below, by imposing that the flow enter and exit each node at most once, and that the flow crosses each edge in only one direction. We refer to these as mutual-exclusion constraints.

Minimize

```
obj: x_ab + x_ba + x_ae + x_ea + x_ad + x_da + x_bc + x_cb + x_be + x_eb + x_ce + x_ec +  
      x_cf + x_fc + x_de + x_ed + x_ef + x_fe
```

Subject to

```
\ mutual exclusion of directional indicators on the same link
```

```
ab_link: x_ab + x_ba <= 1
```

```
ae_link: x_ae + x_ea <= 1
```

```
ad_link: x_ad + x_da <= 1
```

```
bc_link: x_bc + x_cb <= 1
```

```
be_link: x_be + x_eb <= 1
```

```
ce_link: x_ce + x_ec <= 1
```

```
cf_link: x_cf + x_fc <= 1
```

```
de_link: x_de + x_ed <= 1
```

```
ef_link: x_ef + x_fe <= 1
```

```
\ mutual exclusion of incoming into same node
```

```
in_a: x_da + x_ea + x_ba <= 1
```

```
in_b: x_ab + x_eb + x_cb <= 1
```

```
in_c: x_bc + x_ec + x_fc <= 1
```

```
in_d: x_ad + x_ed <= 1
```

```
in_e: x_de + x_ae + x_be + x_ce <= 1
```

```
in_f: x_ef + x_cf <= 1
```

```
\ mutual exclusion of outgoing out of same node
```

```
out_a: x_ad + x_ae + x_ab <= 1
```

```
out_b: x_ba + x_be + x_bc <= 1
```

```

out_c: x_cb + x_ce + x_cf <= 1
out_d: x_da + x_de <= 1
out_e: x_ed + x_ea + x_eb + x_ec <= 1
out_f: x_fe + x_fc <= 1

\ flow balance (flow x: a --> f)
flow_a: x_da - x_ad + x_ea - x_ae + x_ba - x_ab = -1
flow_b: x_ab - x_ba + x_eb - x_be + x_cb - x_bc = 0
flow_c: x_bc - x_cb + x_ec - x_ce + x_fc - x_cf = 0
flow_d: x_ad - x_da + x_ed - x_de = 0
flow_e: x_de - x_ed + x_ae - x_ea + x_be - x_eb + x_ce - x_ec = 0
flow_f: x_ef - x_fe + x_cf - x_fc = 1

\ link capacities (flow demand: x = 4)
ab: 4 x_ab + 4 x_ba <= 7
ae: 4 x_ae + 4 x_ea <= 3
ad: 4 x_ad + 4 x_da <= 6
bc: 4 x_bc + 4 x_cb <= 10
be: 4 x_be + 4 x_eb <= 7
ce: 4 x_ce + 4 x_ec <= 10
cf: 4 x_cf + 4 x_fc <= 10
de: 4 x_de + 4 x_ed <= 5
ef: 4 x_ef + 4 x_fe <= 4

```

Binary

```

x_ab
x_ba
x_ae
x_ea
x_ad
x_da
x_bc
x_cb
x_be
x_eb
x_ce
x_ec
x_cf
x_fc
x_de
x_ed
x_ef
x_fe

```

End

We can then extend the single, whole-flow model with *multiple* whole flows, again in a similar fashion as with multi-commodity real-valued flows. In the next example we do that with an objective function that minimizes the total number of hops, which once again corresponds to routing on shortest paths (compatible with capacities).

Minimize

```

obj: x_ab + x_ba + x_ae + x_ea + x_ad + x_da + x_bc + x_cb + x_be + x_eb + x_ce + x_ec +
      x_cf + x_fc + x_de + x_ed + x_ef + x_fe + y_ab + y_ba + y_ae + y_ea + y_ad + y_da +
      y_bc + y_cb + y_be + y_eb + y_ce + y_ec + y_cf + y_fc + y_de + y_ed + y_ef + y_fe

```

Subject to


```

\ mutual exclusion of flows (directional) on the same link,
ab_link_x: x_ab + x_ba <= 1
ae_link_x: x_ae + x_ea <= 1
ad_link_x: x_ad + x_da <= 1
bc_link_x: x_bc + x_cb <= 1
be_link_x: x_be + x_eb <= 1
ce_link_x: x_ce + x_ec <= 1
cf_link_x: x_cf + x_fc <= 1
de_link_x: x_de + x_ed <= 1
ef_link_x: x_ef + x_fe <= 1

ab_link_y: y_ab + y_ba <= 1
ae_link_y: y_ae + y_ea <= 1
ad_link_y: y_ad + y_da <= 1
bc_link_y: y_bc + y_cb <= 1
be_link_y: y_be + y_eb <= 1
ce_link_y: y_ce + y_ec <= 1
cf_link_y: y_cf + y_fc <= 1
de_link_y: y_de + y_ed <= 1
ef_link_y: y_ef + y_fe <= 1

\ mutual exclusion of incoming flows, for each flow, node
in_a_x: x_da + x_ea + x_ba <= 1
in_b_x: x_ab + x_eb + x_cb <= 1
in_c_x: x_bc + x_ec + x_fc <= 1
in_d_x: x_ad + x_ed <= 1
in_e_x: x_de + x_ae + x_be + x_ce <= 1
in_f_x: x_ef + x_cf <= 1

in_a_y: y_da + y_ea + y_ba <= 1
in_b_y: y_ab + y_eb + y_cb <= 1
in_c_y: y_bc + y_ec + y_fc <= 1
in_d_y: y_ad + y_ed <= 1
in_e_y: y_de + y_ae + y_be + y_ce <= 1
in_f_y: y_ef + y_cf <= 1

\ mutual exclusion of outgoing flows, for each flow, node
out_a_x: x_ad + x_ae + x_ab <= 1
out_b_x: x_ba + x_be + x_bc <= 1
out_c_x: x_cb + x_ce + x_cf <= 1
out_d_x: x_da + x_de <= 1
out_e_x: x_ed + x_ea + x_eb + x_ec <= 1
out_f_x: x_fe + x_fc <= 1

out_a_y: y_ad + y_ae + y_ab <= 1
out_b_y: y_ba + y_be + y_bc <= 1
out_c_y: y_cb + y_ce + y_cf <= 1
out_d_y: y_da + y_de <= 1
out_e_y: y_ed + y_ea + y_eb + y_ec <= 1
out_f_y: y_fe + y_fc <= 1

\ flow-balance constraints (flow x: a --> f, flow y: c --> d)
flow_a_x: x_da - x_ad + x_ea - x_ae + x_ba - x_ab = -1
flow_b_x: x_ab - x_ba + x_eb - x_be + x_cb - x_bc = 0
flow_c_x: x_bc - x_cb + x_ec - x_ce + x_fc - x_cf = 0

```

```

flow_d_x: x_ad - x_da + x_ed - x_de = 0
flow_e_x: x_de - x_ed + x_ae - x_ea + x_be - x_eb + x_ce - x_ec = 0
flow_f_x: x_ef - x_fe + x_cf - x_fc = 1

flow_a_y: y_da - y_ad + y_ea - y_ae + y_ba - y_ab = 0
flow_b_y: y_ab - y_ba + y_eb - y_be + y_cb - y_bc = 0
flow_c_y: y_bc - y_cb + y_ec - y_ce + y_fc - y_cf = 1
flow_d_y: y_ad - y_da + y_ed - y_de = -1
flow_e_y: y_de - y_ed + y_ae - y_ea + y_be - y_eb + y_ce - y_ec = 0
flow_f_y: y_ef - y_fe + y_cf - y_fc = 0

\ link-capacity constraints (flow demands: x = 4, y = 3)
ab_c: 4 x_ab + 4 x_ba + 3 y_ab + 3 y_ba <= 7
ae_c: 4 x_ae + 4 x_ea + 3 y_ae + 3 y_ea <= 3
ad_c: 4 x_ad + 4 x_da + 3 y_ad + 3 y_da <= 6
bc_c: 4 x_bc + 4 x_cb + 3 y_bc + 3 y_cb <= 10
be_c: 4 x_be + 4 x_eb + 3 y_be + 3 y_eb <= 7
ce_c: 4 x_ce + 4 x_ec + 3 y_ce + 3 y_ec <= 10
cf_c: 4 x_cf + 4 x_fc + 3 y_cf + 3 y_fc <= 10
de_c: 4 x_de + 4 x_ed + 3 y_de + 3 y_ed <= 5
ef_c: 4 x_ef + 4 x_fe + 3 y_ef + 3 y_fe <= 4

```

Binary

```

x_ab
x_ba
x_ae
x_ea
x_ad
x_da
x_bc
x_cb
x_be
x_eb
x_ce
x_ec
x_cf
x_fc
x_de
x_ed
x_ef
x_fe

y_ab
y_ba
y_ae
y_ea
y_ad
y_da
y_bc
y_cb
y_be
y_eb
y_ce
y_ec
y_cf
y_fc

```

```

y_de
y_ed
y_ef
y_fe
End

```

The same model seen so far for whole flows can also be changed with various optimization goals. For example, if instead of the total hop count we are interested in minimizing the maximal link usage, we can do that exactly as we did for the fractional flows.

Other optimization goals might be a bit more complicated. For example, what if we want to maximize the flow intensities? We might want to maximize the total flow intensity, or for fairness the minimal flow intensity. Once again, the trick is to add more control variables, and a natural approach would be to model intensity of each flow as a control variable. For example, we could say that r_x is the intensity of flow x . However, we would then have to write the capacity constraints as $r_x x_{uv} + r_x x_{vu} \leq C(u, v)$, which we can not do, since the product of control variables $r_x x_{uv}$ is not a linear expression of the control variables.

How can we get around this modeling obstacle? Once again—you guessed it—we add more variables. A straightforward approach is to use the same real-valued variables we used in the basic fractional-flow model. Thus for each flow x and for each edge (u, v) we define rx_{uv} as positive, real-value variables. We then write a constraint $rx_{uv} \leq C(u, v)x_{uv}$ that effectively sets rx_{uv} to zero if the corresponding indicator variable x_{uv} is zero, or at most the capacity $C(u, v)$ if x_{uv} is one.

One last but essential point about the combination of real and indicator variables. If we use this model with only the simple constraints for the indicator variables, namely only those that define flow-balance constraints, then we get bad results. Recall from the example above that the flow-balance constraints alone would allow for a flow to circle back to a node, resulting in multiple entry and exit edges. Therefore, it is possible to have, for example, $x_{ab} = 1$ and $x_{ae} = 1$. In combination with the real-value flow intensity variables, this would allow the solver to effectively split the x flow between the (a, b) and (a, e) edges, which is not what we want. We must therefore add the more restrictive constraints that force the input and output indicator variables for each node (and each flow) to be mutually exclusive. The resulting model is shown below:

```

Maximize
obj: m

Subject to
\ m = min {lambda_x, lambda_y}, or more precisely m <= min {lambda_x, lambda_y}
x_min: lambda_x - m >= 0
y_min: lambda_y - m >= 0

\ flow balance for real-value rate variables
flow_a_rx: rx_da - rx_ad + rx_ea - rx_ae + rx_ba - rx_ab + lambda_x = 0
flow_b_rx: rx_ab - rx_ba + rx_eb - rx_be + rx_cb - rx_bc = 0
flow_c_rx: rx_bc - rx_cb + rx_ec - rx_ce + rx_fc - rx_cf = 0
flow_d_rx: rx_ad - rx_da + rx_ed - rx_de = 0
flow_e_rx: rx_de - rx_ed + rx_ae - rx_ea + rx_be - rx_eb + rx_ce - rx_ec = 0
flow_f_rx: rx_ef - rx_fe + rx_cf - rx_fc - lambda_x = 0

flow_a_ry: ry_da - ry_ad + ry_ea - ry_ae + ry_ba - ry_ab = 0
flow_b_ry: ry_ab - ry_ba + ry_eb - ry_be + ry_cb - ry_bc = 0
flow_c_ry: ry_bc - ry_cb + ry_ec - ry_ce + ry_fc - ry_cf - lambda_y = 0
flow_d_ry: ry_da - ry_ad + ry_ed - ry_de + lambda_y = 0
flow_e_ry: ry_de - ry_ed + ry_ae - ry_ea + ry_be - ry_eb + ry_ce - ry_ec = 0
flow_f_ry: ry_ef - ry_fe + ry_cf - ry_fc = 0

```

```

\ flow balance for indicators, for each flow, for each node
flow_a_x: x_da - x_ad + x_ea - x_ae + x_ba - x_ab = -1
flow_b_x: x_ab - x_ba + x_eb - x_be + x_cb - x_bc = 0
flow_c_x: x_bc - x_cb + x_ec - x_ce + x_fc - x_cf = 0
flow_d_x: x_ad - x_da + x_ed - x_de = 0
flow_e_x: x_de - x_ed + x_ae - x_ea + x_be - x_eb + x_ce - x_ec = 0
flow_f_x: x_ef - x_fe + x_cf - x_fc = 1

flow_a_y: y_da - y_ad + y_ea - y_ae + y_ba - y_ab = 0
flow_b_y: y_ab - y_ba + y_eb - y_be + y_cb - y_bc = 0
flow_c_y: y_bc - y_cb + y_ec - y_ce + y_fc - y_cf = 1
flow_d_y: y_ad - y_da + y_ed - y_de = -1
flow_e_y: y_de - y_ed + y_ae - y_ea + y_be - y_eb + y_ce - y_ec = 0
flow_f_y: y_ef - y_fe + y_cf - y_fc = 0

\ mutual exclusion of incoming into same node, for each flow, for each node
in_a_x: x_da + x_ea + x_ba <= 1
in_b_x: x_ab + x_eb + x_cb <= 1
in_c_x: x_bc + x_ec + x_fc <= 1
in_d_x: x_ad + x_ed <= 1
in_e_x: x_de + x_ae + x_be + x_ce <= 1
in_f_x: x_ef + x_cf <= 1

in_a_y: y_da + y_ea + y_ba <= 1
in_b_y: y_ab + y_eb + y_cb <= 1
in_c_y: y_bc + y_ec + y_fc <= 1
in_d_y: y_ad + y_ed <= 1
in_e_y: y_de + y_ae + y_be + y_ce <= 1
in_f_y: y_ef + y_cf <= 1

\ mutual exclusion of outgoing out of same node, for each flow, for each node
out_a_x: x_ad + x_ae + x_ab <= 1
out_b_x: x_ba + x_be + x_bc <= 1
out_c_x: x_cb + x_ce + x_cf <= 1
out_d_x: x_da + x_de <= 1
out_e_x: x_ed + x_ea + x_eb + x_ec <= 1
out_f_x: x_fe + x_fc <= 1

out_a_y: y_ad + y_ae + y_ab <= 1
out_b_y: y_ba + y_be + y_bc <= 1
out_c_y: y_cb + y_ce + y_cf <= 1
out_d_y: y_da + y_de <= 1
out_e_y: y_ed + y_ea + y_eb + y_ec <= 1
out_f_y: y_fe + y_fc <= 1

\ link capacities, for each link
ab_c: rx_ab + rx_ba + ry_ab + ry_ba <= 7
ae_c: rx_ae + rx_ea + ry_ae + ry_ea <= 3
ad_c: rx_ad + rx_da + ry_ad + ry_da <= 6
bc_c: rx_bc + rx_cb + ry_bc + ry_cb <= 10
be_c: rx_be + rx_eb + ry_be + ry_eb <= 7
ce_c: rx_ce + rx_ec + ry_ce + ry_ec <= 10
cf_c: rx_cf + rx_fc + ry_cf + ry_fc <= 10
de_c: rx_de + rx_ed + ry_de + ry_ed <= 5
ef_c: rx_ef + rx_fe + ry_ef + ry_fe <= 4

```

\ control of real-value flow variables by corresponding indicators, for each flow and link

ab_cx: rx_ab - 7 x_ab <= 0
ae_cx: rx_ae - 3 x_ae <= 0
ad_cx: rx_ad - 6 x_ad <= 0
bc_cx: rx_bc - 10 x_bc <= 0
be_cx: rx_be - 7 x_be <= 0
ce_cx: rx_ce - 10 x_ce <= 0
cf_cx: rx_cf - 10 x_cf <= 0
de_cx: rx_de - 5 x_de <= 0
ef_cx: rx_ef - 4 x_ef <= 0

ba_cx: rx_ba - 7 x_ba <= 0
ea_cx: rx_ea - 3 x_ea <= 0
da_cx: rx_da - 6 x_da <= 0
cb_cx: rx_cb - 10 x_cb <= 0
eb_cx: rx_eb - 7 x_eb <= 0
ec_cx: rx_ec - 10 x_ec <= 0
fc_cx: rx_fc - 10 x_fc <= 0
ed_cx: rx_ed - 5 x_ed <= 0
fe_cx: rx_fe - 4 x_fe <= 0

ab_cy: ry_ab - 7 y_ab <= 0
ae_cy: ry_ae - 3 y_ae <= 0
ad_cy: ry_ad - 6 y_ad <= 0
bc_cy: ry_bc - 10 y_bc <= 0
be_cy: ry_be - 7 y_be <= 0
ce_cy: ry_ce - 10 y_ce <= 0
cf_cy: ry_cf - 10 y_cf <= 0
de_cy: ry_de - 5 y_de <= 0
ef_cy: ry_ef - 4 y_ef <= 0

ba_cy: ry_ba - 7 y_ba <= 0
ea_cy: ry_ea - 3 y_ea <= 0
da_cy: ry_da - 6 y_da <= 0
cb_cy: ry_cb - 10 y_cb <= 0
eb_cy: ry_eb - 7 y_eb <= 0
ec_cy: ry_ec - 10 y_ec <= 0
fc_cy: ry_fc - 10 y_fc <= 0
ed_cy: ry_ed - 5 y_ed <= 0
fe_cy: ry_fe - 4 y_fe <= 0

Binary

x_ab
x_ba
x_ae
x_ea
x_ad
x_da
x_bc
x_cb
x_be
x_eb
x_ce
x_ec

x_cf
x_fc
x_de
x_ed
x_ef
x_fe

y_ab
y_ba
y_ae
y_ea
y_ad
y_da
y_bc
y_cb
y_be
y_eb
y_ce
y_ec
y_cf
y_fc
y_de
y_ed
y_ef
y_fe

End